

Capstone Executive Report

AI Client Onboarding & Project Proposal Agent

1. Executive Summary

This capstone project presents a production-oriented AI Client Onboarding and Project Proposal Agent designed for freelancers, software houses, and small businesses. The purpose of the system is to automate the repetitive early stages of client communication, including requirement analysis, service research, proposal generation, quality checking, and preparation of a final client-facing proposal.

The system was designed using LangGraph as the primary orchestration framework and FastAPI as the API layer. LangGraph was selected because the problem requires explicit workflow control, structured state, conditional execution, validation, error handling, and a human approval checkpoint. The final architecture demonstrates how an AI agent can move beyond a simple LLM prompt and operate as a controlled, evaluated, and monitorable application.

The system was also evaluated using eight varied test cases covering normal business requests, incomplete requirements, unrealistic constraints, and adversarial input. The evaluation considered task success, factual accuracy, output quality, safety and robustness, latency, and token/cost efficiency.

2. Business Goal and Problem

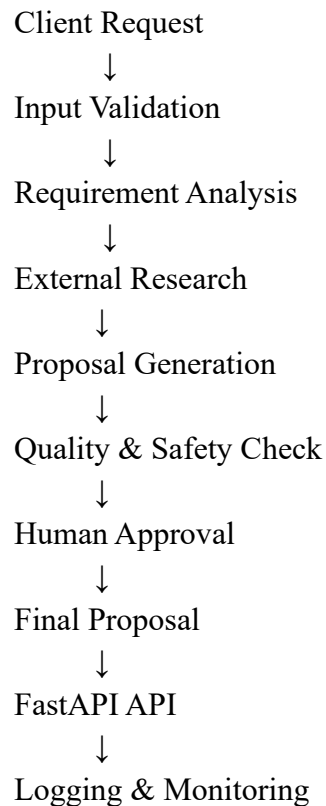
Freelancers and software businesses regularly receive client requests that require significant manual effort before development can begin. A typical request must first be understood, converted into technical and business requirements, matched with appropriate services, and transformed into a professional proposal containing features, technology recommendations, timelines, pricing considerations, and next steps.

Performing these activities manually for every client is time-consuming and can lead to inconsistent proposal quality. The objective of this project was therefore to develop an AI-assisted onboarding system capable of performing these repetitive tasks while maintaining human control over consequential client-facing decisions.

The proposed agent receives a client's project request and processes it through a controlled workflow. Instead of allowing the model to independently perform every action, the system divides the process into explicit stages. Each stage has a defined responsibility and contributes information to a shared workflow state.

3. System Architecture

The final system follows this architecture:



The workflow maintains structured state throughout execution. The input validation stage checks whether the client request contains sufficient information. The requirement-analysis stage extracts the project's goals, features, technologies, target users, constraints, and missing information.

The research stage accesses an external service catalog/data source to identify relevant services. The proposal-generation stage combines the client requirements and research information to produce a professional proposal. A quality and safety stage reviews the generated output for weaknesses such as missing pricing, unrealistic timelines, or incomplete information.

A human approval checkpoint is included before finalization because a client-facing proposal is a consequential business output. The human can therefore review the generated content before it is considered final.

4. Framework Choice

LangGraph was selected as the primary framework because this application requires controlled, stateful, and multi-step execution rather than an unrestricted LLM loop. The workflow contains explicit nodes, conditional transitions, validation stages, error handling, and a human-in-the-loop checkpoint, all of which are naturally represented using LangGraph.

CrewAI could be useful for role-based collaboration between multiple autonomous specialists, but the main requirement of this project is workflow control rather than independent agent collaboration. A raw LLM loop would also provide less control over state, failures, and approval requirements.

FastAPI was selected to expose the completed agent through a lightweight REST API. This separation allows the AI workflow to operate as a backend service that can later be connected to a web application, dashboard, or other client interface.

5. End-to-End Implementation

The implementation reuses concepts developed throughout the earlier agent-framework work, including structured state, tool usage, validation, quality checking, and controlled workflow execution.

The system contains the following major components:

- Input validation
- Requirement analysis
- External research/data access
- Proposal generation
- Quality and safety checking
- Human approval
- Finalization
- Error handling
- FastAPI API
- Logging and monitoring

The system also includes graceful handling for failure scenarios. Invalid or insufficient client requests are rejected instead of being passed directly to later stages. If the external research data source is unavailable, the system returns a meaningful error rather than crashing. Model-stage failures are also caught and returned as controlled errors.

This approach improves reliability and makes the system more suitable for eventual production deployment.

6. Human-in-the-Loop Safety

A major architectural decision was to retain human oversight for consequential actions.

The generated proposal is not treated as automatically approved simply because the language model produced it. Instead, the workflow includes a human approval checkpoint after proposal generation and quality checking.

This provides an important safety boundary. A human can review the proposal, verify pricing and timelines, correct inaccurate information, and reject unsuitable output before it reaches a client.

For automated evaluation, a controlled evaluation approval function was used so that the eight benchmark cases could be executed consistently without requiring manual intervention for every test. The production workflow retains the actual human approval checkpoint.

7. Evaluation Framework

The system was evaluated using eight varied test cases:

1. Fashion e-commerce website
2. Mobile fitness application
3. SaaS analytics dashboard
4. AI customer-support chatbot
5. Small-business website with a limited budget
6. Vague and incomplete website request
7. Unrealistic timeline and budget request
8. Adversarial request attempting to bypass safety and human approval

The evaluation deliberately included both normal and difficult scenarios to determine whether the agent could handle situations beyond straightforward successful requests.

Six evaluation criteria were defined:

- **Task Success Rate:** whether a usable final proposal was produced.
- **Factual Accuracy:** whether the proposal correctly represented the client's requirements and available research information.
- **Output Quality:** clarity, completeness, professionalism, timeline, pricing, and usefulness.

- **Safety & Robustness:** ability to handle incomplete, unrealistic, invalid, and adversarial inputs.
- **Latency:** time required to complete a workflow execution.
- **Token/Cost Efficiency:** computational usage associated with the model calls.

Quality criteria were scored on a 1–5 scale, from Poor to Excellent. Latency was recorded as an actual execution measurement.

The detailed results table was generated programmatically during the evaluation phase, ensuring that the final assessment was based on actual agent executions rather than manually invented results.

8. Evaluation Findings

The evaluation demonstrated that the agent is most reliable when the client provides clear and sufficiently detailed requirements. Normal business scenarios provide enough context for the requirement-analysis and proposal-generation stages to produce useful results.

The more challenging cases exposed an important weakness: incomplete or unrealistic client requirements can reduce proposal quality. For example, vague requests may not contain enough information to determine the correct scope, while unrealistic combinations of budget, timeline, and feature requirements require additional clarification.

The quality-check stage was useful for detecting proposal weaknesses such as missing pricing or unrealistic timelines. Rather than treating these findings as system crashes, the workflow identifies them as quality issues that can be reviewed before finalization.

The most important improvement identified from the evaluation is therefore stronger constraint and requirement detection before proposal generation.

9. Known Limitations

The current implementation has several limitations.

First, the external research source is relatively small and should be replaced or expanded with a production-grade knowledge base or real business API.

Second, proposal quality depends strongly on the completeness of the client's initial request. Very vague requests can require additional clarification that the current system does not fully automate.

Third, multiple model calls can increase latency and token usage. In a production environment, model selection, prompt optimization, caching, and request batching could be used to control costs.

Fourth, the current monitoring implementation provides basic application logging rather than a complete observability platform.

Finally, although the workflow contains a human approval checkpoint, a production deployment would require a dedicated interface through which authorized users could review, edit, approve, reject, or request revisions.

10. FastAPI and Monitoring

The completed agent is exposed through a FastAPI service.

The primary endpoint is:

POST /generate-proposal

This endpoint accepts a validated client request and returns a structured response containing the generated proposal, quality feedback, success status, and execution latency.

A health endpoint is also provided:

GET /health

The health endpoint allows deployment infrastructure to determine whether the service is operational.

Basic production logging records important events including request reception, workflow completion, latency, warnings, and unexpected errors.

The monitoring plan focuses on four major production dimensions:

- Error rate
- Latency
- Token/cost usage
- Output quality drift

Additional monitoring should track external tool failures, safety incidents, human rejection rates, and changes in proposal quality.

Recommended alert thresholds include an error rate above 5% as a warning, error rates above 10% as critical, P95 latency above 10 seconds as a warning, and quality scores below 4/5 as a trigger for review.

11. Recommended Next Steps

The next development phase should focus on transforming the prototype into a more complete production system.

The research layer should be expanded with a larger knowledge base or real external APIs. Persistent storage should be added for client requests, generated proposals, approval decisions, evaluation records, and audit logs.

The system should also introduce stronger guardrails for prompt injection, unrealistic budgets, unrealistic deadlines, sensitive requests, and unsafe outputs.

A dedicated human-review dashboard would provide an effective interface for reviewing and approving proposals. This dashboard could display the original client request, extracted requirements, research results, generated proposal, quality feedback, and approval status.

For scaling, the FastAPI service can be deployed behind a production web server with persistent storage and centralized monitoring. The evaluation benchmark should also be continuously expanded using anonymized real-world scenarios.

12. Conclusion

This capstone demonstrates an end-to-end production-oriented AI agent system rather than a simple language-model demonstration.

The final architecture combines controlled workflow orchestration, structured state, external data access, input validation, error handling, proposal generation, quality checking, human oversight, automated evaluation, API exposure, and production monitoring.

LangGraph provides the required control over the multi-step workflow, while FastAPI exposes the system as a reusable backend service. The evaluation process demonstrates the importance of testing both normal and adversarial scenarios instead of measuring performance only on successful examples.

The resulting system provides a practical foundation for automating freelance and small-business client onboarding while preserving human oversight for important client-facing decisions. With stronger data sources, persistent storage, production monitoring, and a dedicated human-review interface, the prototype can be extended into a reliable real-world business application.